DEPI SOFTWARE TESTING TRACK — FINAL PROJECT

Project Proposal

Executive Summary, Problem Statement, Objectives, Proposed Solution, Project Scope, and Implementation Methodology

Project	Banking Transaction System	**Document**	Project Proposal
Prepared By	Aghapy Antonious (Backend Services & Admin)	**Reviewed By**	Project Team

Part 1. Executive Summary

High-level overview of the Banking Transaction System, its core value proposition, and target users.

The Banking Transaction System is a secure, reliable, full-stack web-based application designed to automate core retail banking operations. It empowers customers to open and manage multiple bank account types (Savings, Checking, and Fixed Deposit), execute deposits, withdrawals, and fund transfers with real-time atomic balance updates, and export financial statements. Simultaneously, it provides bank administrators with centralized oversight tools to monitor bank reserves, search customer records, freeze or unfreeze accounts, and inspect immutable security and financial audit logs. The primary objective is to enhance operational efficiency, eliminate manual calculation errors, and deliver a transparent, thoroughly tested financial management platform.

Part 2. Problem Statement & Background

Operational and technical challenges addressed by the platform.

Traditional and manual banking administrative workflows frequently suffer from high operational overhead, slow transaction processing, and human error when calculating monthly savings interest, fixed deposit maturity payouts, or dynamic inter-customer transfer fees. Furthermore, concurrent financial requests without strict transactional locking can cause balance inconsistencies or lost updates. Customers require a fast, transparent, 24/7 digital platform to execute transfers, verify fee quotes upfront, and track running balances, while bank administrators require centralized visibility over account security statuses and compliance audit trails without compromising customer privacy or least-privilege security boundaries.

Part 3. Objectives & Proposed Solution

Strategic goals and architectural design of the proposed banking platform.

3.1 Core Project Objectives

Objective Pillar	Target Outcome & Operational Impact
Operational Efficiency	Streamline core banking operations and eliminate manual calculation errors in transaction processing, dynamic transfer fee computation, and scheduled interest generation through deterministic, automated domain rules.

Objective Pillar	Target Outcome & Operational Impact
Customer Empowerment	Provide a secure, 24/7 self-service web portal and RESTful API for opening accounts, executing instant fund transfers with upfront fee transparency, and downloading paginated financial statements in CSV format.
Administrative Control & Compliance	Empower bank administrators with a dedicated control dashboard and immutable audit logs to monitor bank-wide reserves, investigate security events, and manage account freeze/unfreeze statuses under strict separation of duties.

3.2 Proposed Solution Architecture

Perspective	Architectural & Quality Assurance Approach
Business & Operational Perspective	Delivers a unified full-stack web portal built with Java 17+ and Spring Boot 3.5.x backed by an embedded SQLite database and responsive Thymeleaf user interfaces. The platform bridges customer self-service needs (instant account management, fee quotes, and statements) with administrative governance (reserve metrics, customer search, account freeze controls, and audit trails).
Technical & Quality Perspective	Applies strict Object-Oriented Programming (OOP) principles and layered architecture to model complex account hierarchies (Savings, Checking, and Fixed Deposit) with exact piaster-based monetary storage (BIGINT in SQLite, BigDecimal in Java) and Write-Ahead Logging (WAL) concurrency control. System reliability is verified through automated testing frameworks (JUnit 5, Mockito, Spring MockMvc, Postman, and Selenium 4 WebDriver) alongside structured black-box QA techniques (Equivalence Partitioning, Boundary Value Analysis, and Decision Tables).

Part 4. Project Scope

Clear definition of included capabilities (In-Scope) and excluded boundaries (Out-of-Scope).

4.1 In-Scope Capabilities

Functional Domain	In-Scope Specifications & Business Rules
User & Role Management	Visitor self-registration, stateful customer web authentication, stateless HTTP Basic REST API authentication, BCrypt password hashing, 5-attempt / 15-minute login lockout, 15-minute idle session timeout, and server-side Role-Based Access Control (CUSTOMER and ADMIN).
Account Types & Domain Rules	• Savings Account: Minimum opening and ongoing balance of 1,000.00 EGP with automated monthly interest calculation (17% annual rate / 12, rounded HALF_UP). • Checking Account: 0.00 EGP minimum opening balance with an overdraft facility supported down to -1,000.00 EGP. • Fixed Deposit Account: Time-locked deposits starting at 5,000.00 EGP across 3-month (18%), 6-month (20%), or 12-month (22%) terms with full principal and interest payout upon maturity. • Account Constraints: Unique 12-digit Luhn-validated account numbers; maximum of 5 non-closed accounts per customer.

Functional Domain	In-Scope Specifications & Business Rules
Core Financial Transactions	Cash deposit simulations (10.00 to 100,000.00 EGP), withdrawals, and account-to-account transfers enforcing single-transaction (20,000.00 EGP), daily (50,000.00 EGP), and monthly (200,000.00 EGP) limits. Includes inter-customer transfer fee calculation (0.1%, min 0.50 EGP, max 20.00 EGP), free own-account transfers, idempotency protection, and append-only ledger logging.
Reporting & Auditing	Paginated customer account statements (newest first, with post-transaction running balance), date-range and transaction-type filtering, UTF-8 CSV statement export, and comprehensive security/financial audit logging for administrators.
User Interface & REST API	Responsive Thymeleaf web views with permanent HTML element IDs for Selenium test automation, paired with documented RESTful API endpoints (OpenAPI / Swagger UI).

4.2 Out-of-Scope Boundaries

Excluded Area	Boundary Justification
Physical ATM & Card Networks	No hardware ATM interface integration, POS terminal connectivity, or physical debit/credit card issuance and processing.
Multi-Currency & FX Operations	The system operates strictly in Egyptian Pounds (EGP). Foreign currency accounts and exchange rate conversions are excluded.
External Notification Gateways	No integration with third-party commercial SMS gateways or external SMTP email notification servers.
Native Mobile Applications	No native iOS or Android mobile client development (mobile accessibility is provided via responsive web design).

Part 5. Methodology & Implementation Plan

Agile delivery lifecycle, sprint governance, and quality verification workflow.

The project follows an iterative Agile development methodology across an 8-week schedule, utilizing Jira for sprint task tracking and defect management, and GitHub for source control and mandatory Pull Request peer reviews:

Execution Stage	Core Engineering & Quality Activities
Stage 1: Requirements & Architecture Design	Define stakeholder expectations, user stories, functional/non-functional requirements, business rules, SQLite database schema, and REST API contracts.
Stage 2: Core Full-Stack Development	Implement Spring Boot backend services, domain account hierarchies, Spring Security filter chains, atomic transaction services, and Thymeleaf web templates.
Stage 3: Quality Assurance & Automated Testing	Execute JUnit 5 and Mockito unit tests, MockMvc controller integration tests, Postman API test collections, Selenium 4 WebDriver UI test suites, and manual boundary/decision-table test cases.

Execution Stage	Core Engineering & Quality Activities
Stage 4: Static Analysis & Final Delivery	Perform static code analysis (SonarLint / SpotBugs), OWASP dependency vulnerability scanning, full regression testing, JaCoCo coverage verification, and final presentation rehearsals.